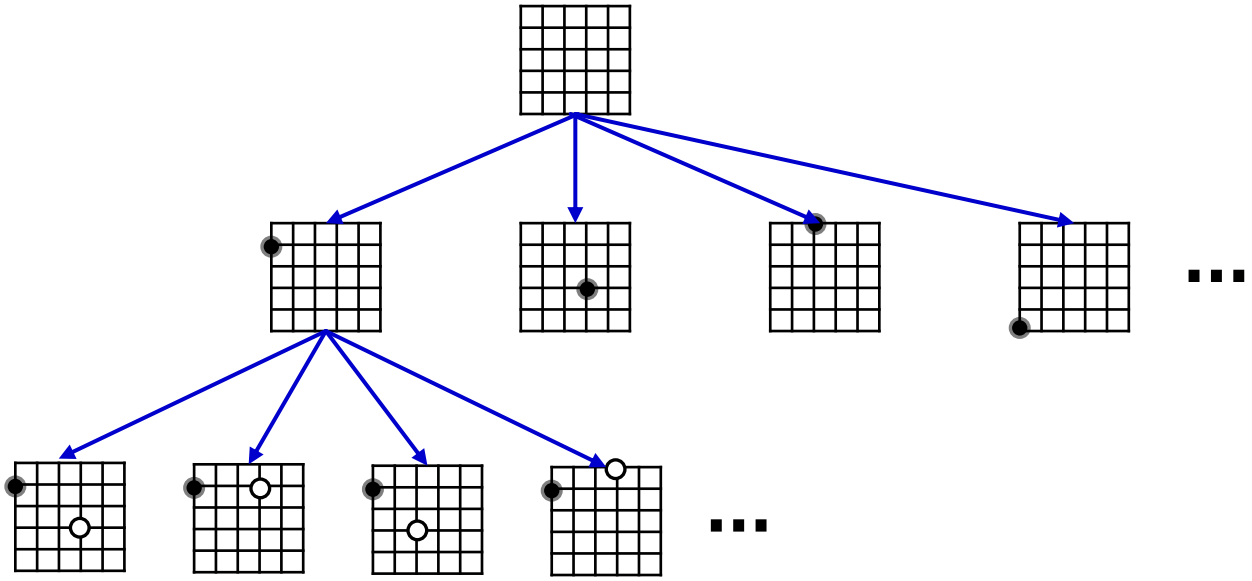
Adversarial Search – Monte Carlo Tree Search

AIMA Chapter 5.4

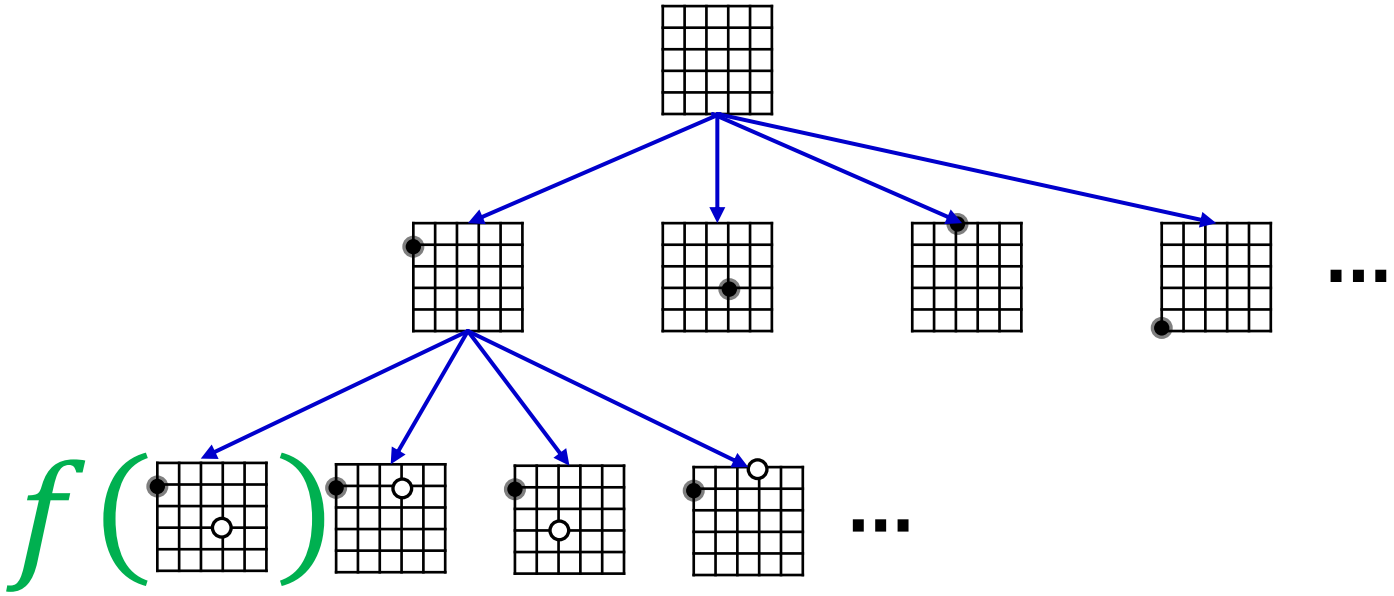


Go has a branching factor of 361 initially.

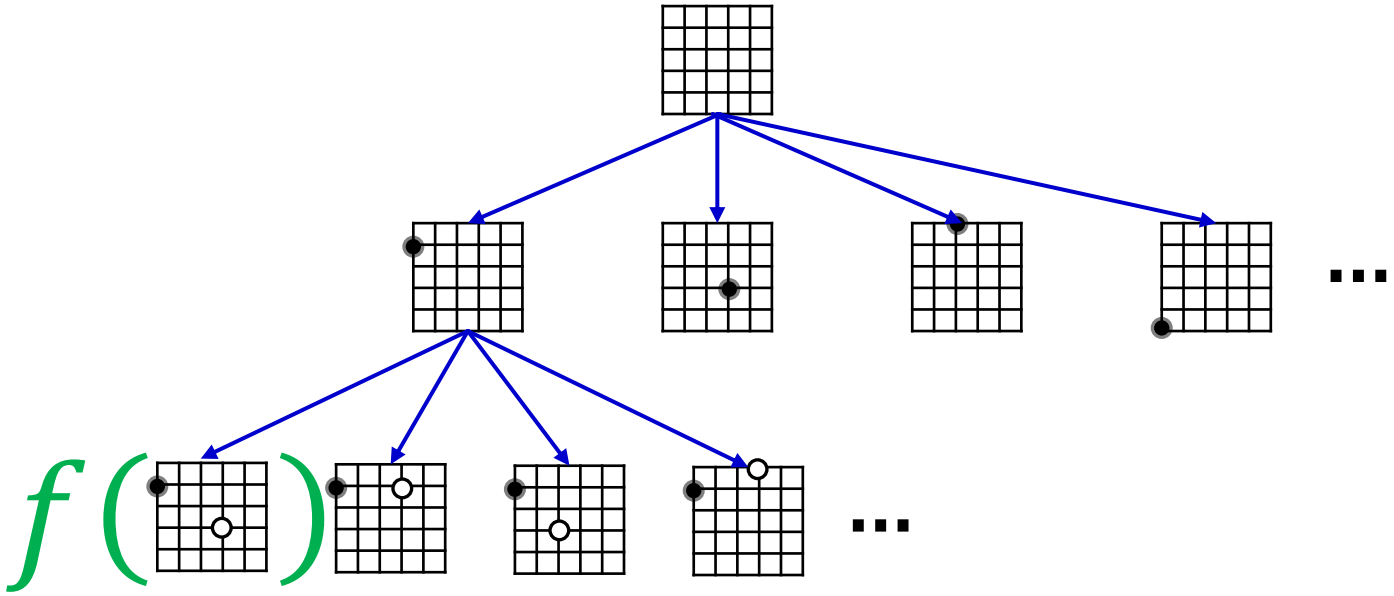
Even if we look four moves ahead, that's

$$361 \times 360 \times 359 \times 358 = 16{,}702{,}719{,}120 \sim 10^{10}$$

intermediate positions to evaluate.



Previously: expand search tree up to a certain depth, then evaluate the non-leaf nodes using some evaluation function $f(n)$.

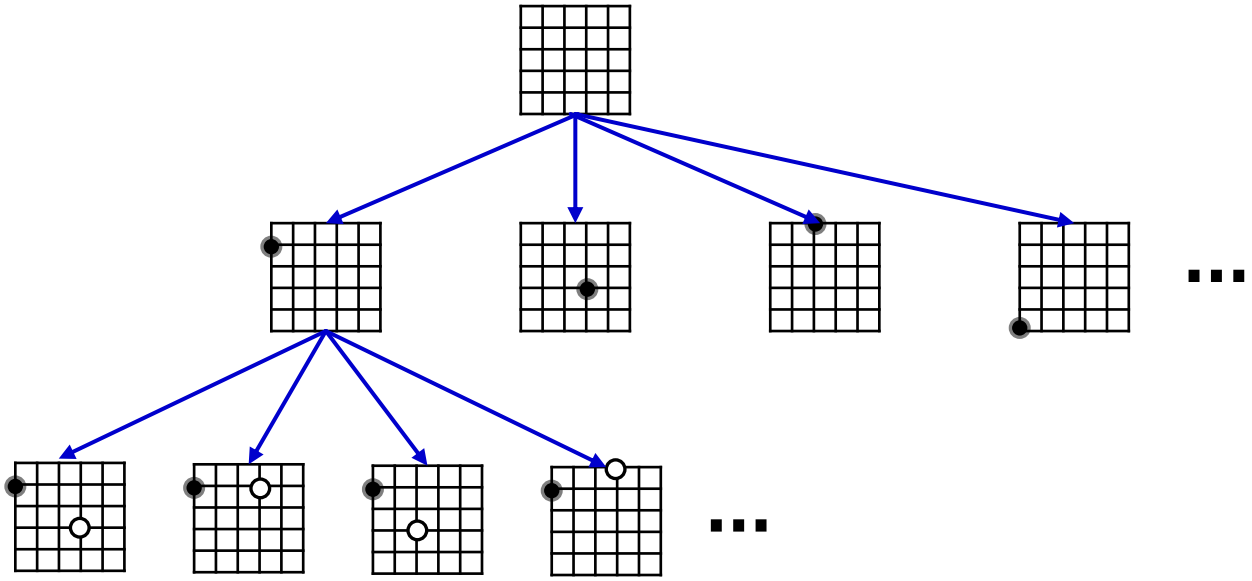


Lots of pitfalls:

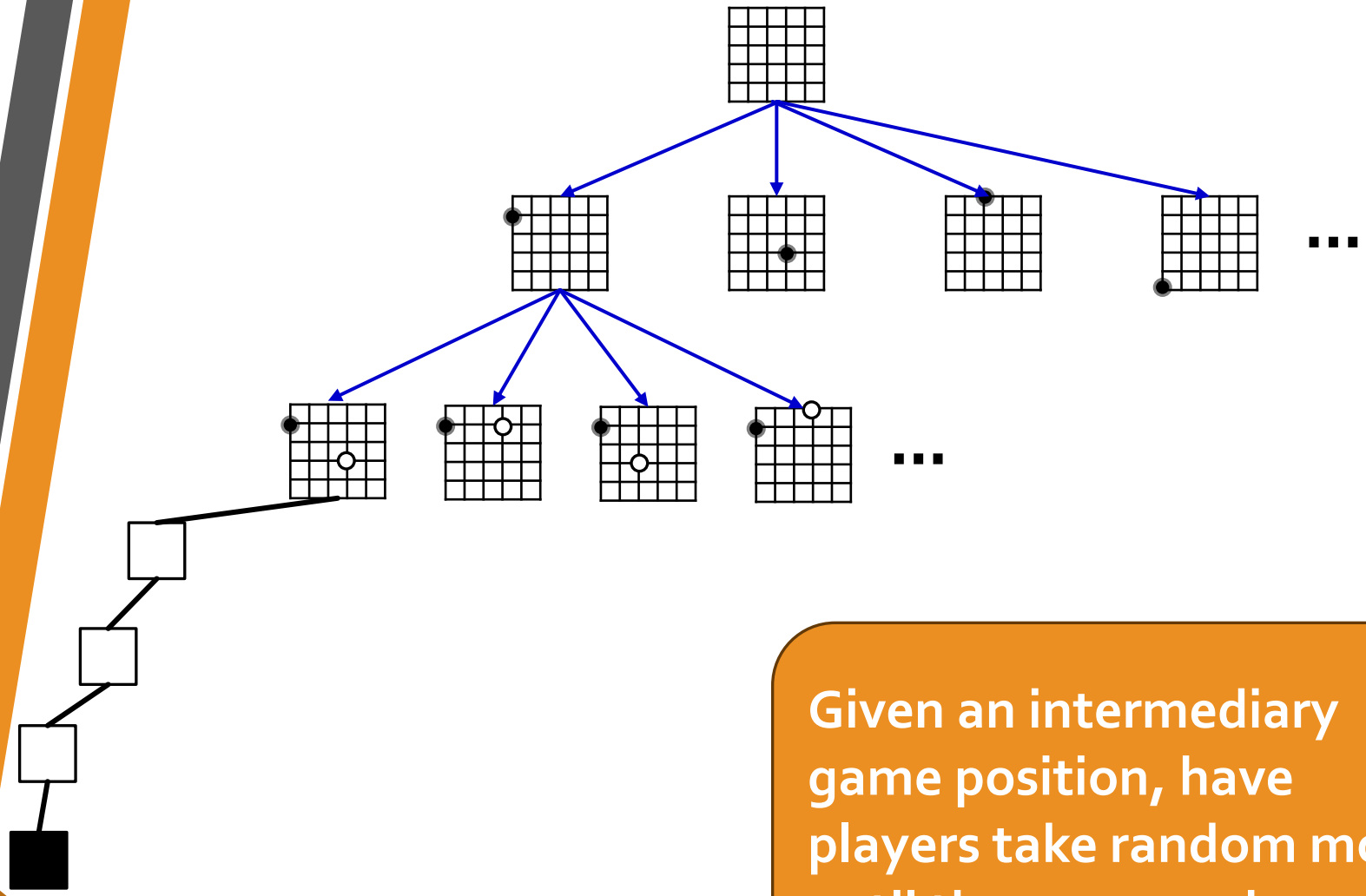
a. What features do we use?

b. What function form makes most sense?

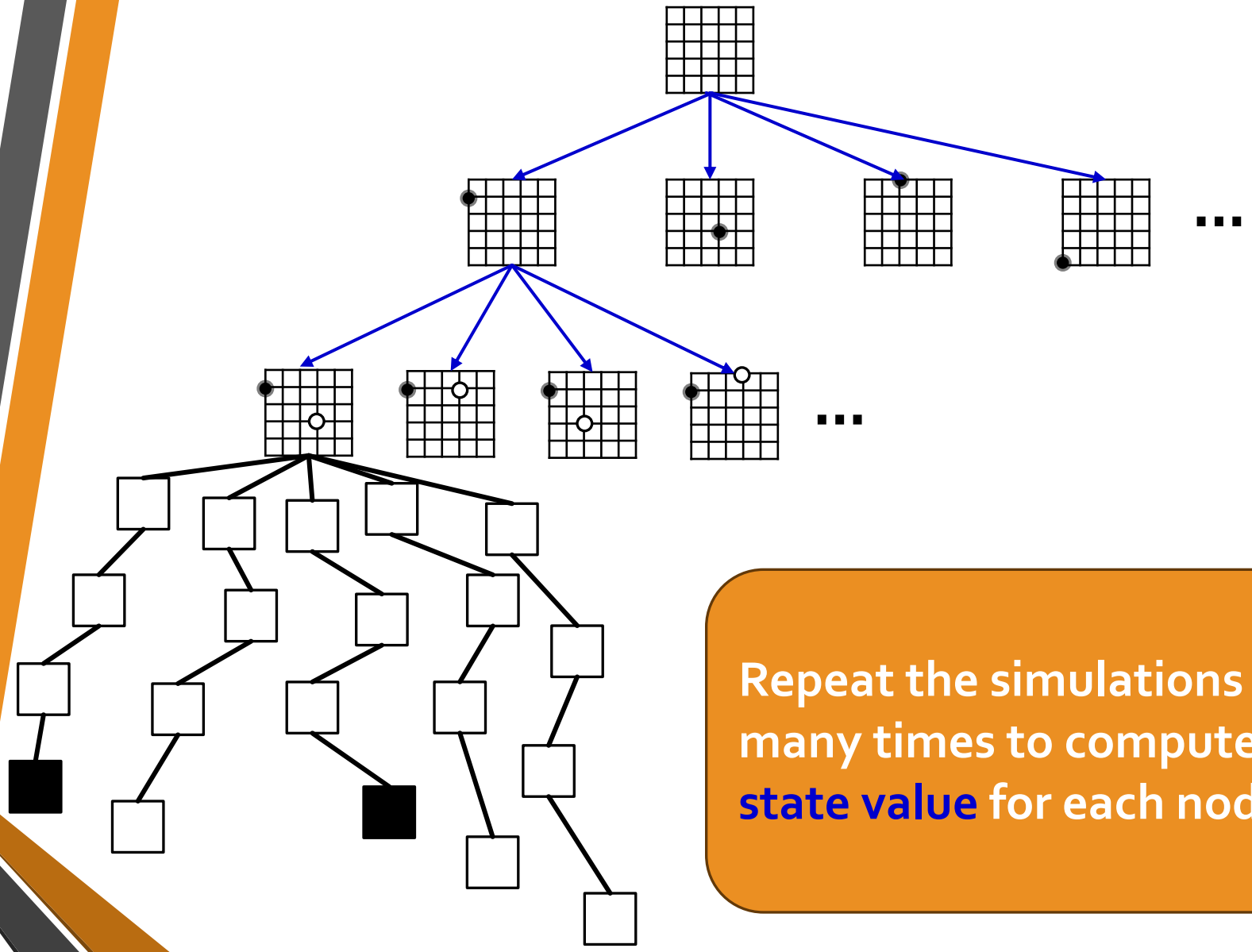
c. Initially promising states may be disastrous later.



Monte-Carlo Tree Search: instead of relying on evaluation function to estimate the value of an intermediate node, use simulations.

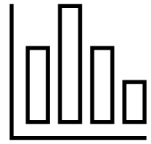


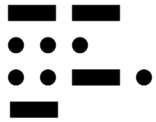
Given an intermediary game position, have players take random moves until the game ends.



Repeat the simulations many times to compute a **state value** for each node.

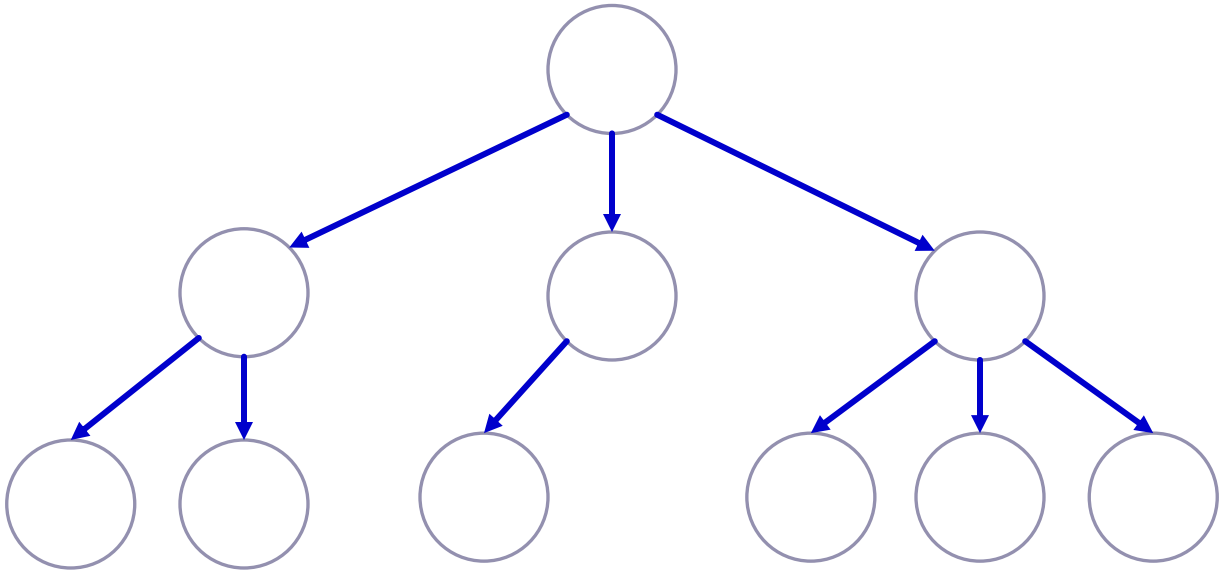
1. **Running simulations:** Which gameplays do we want to simulate? All of them? Uniformly at random?
2. **Storing Information:** how do we evaluate the quality of moves?
3. **Runtime:** how many simulations are enough?





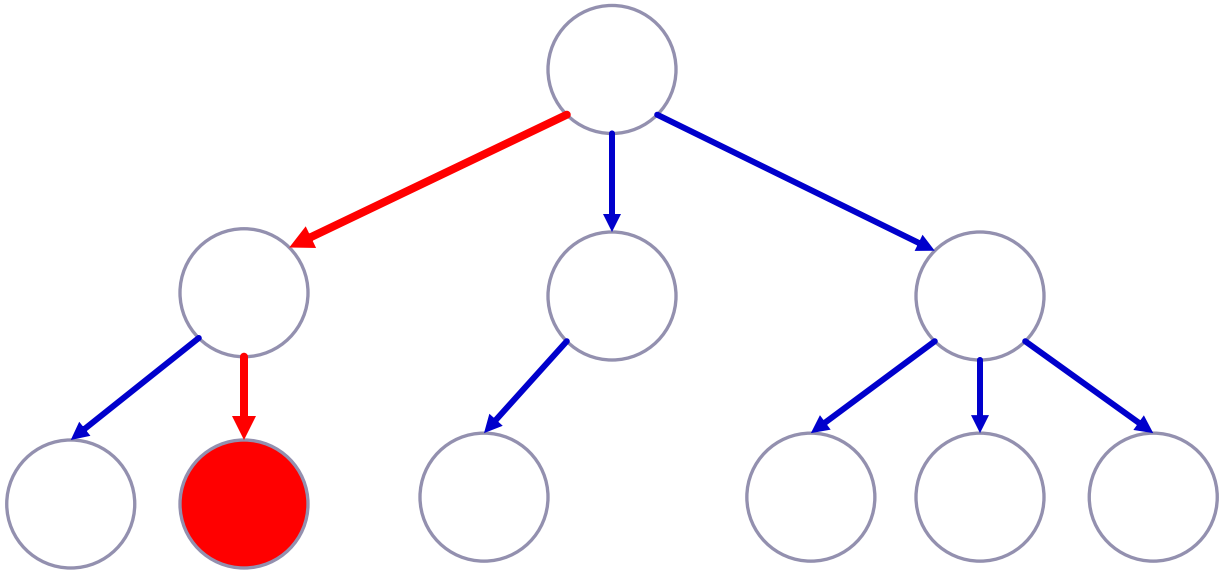
Step 1: Selection

Select a node that we will start the simulation from.



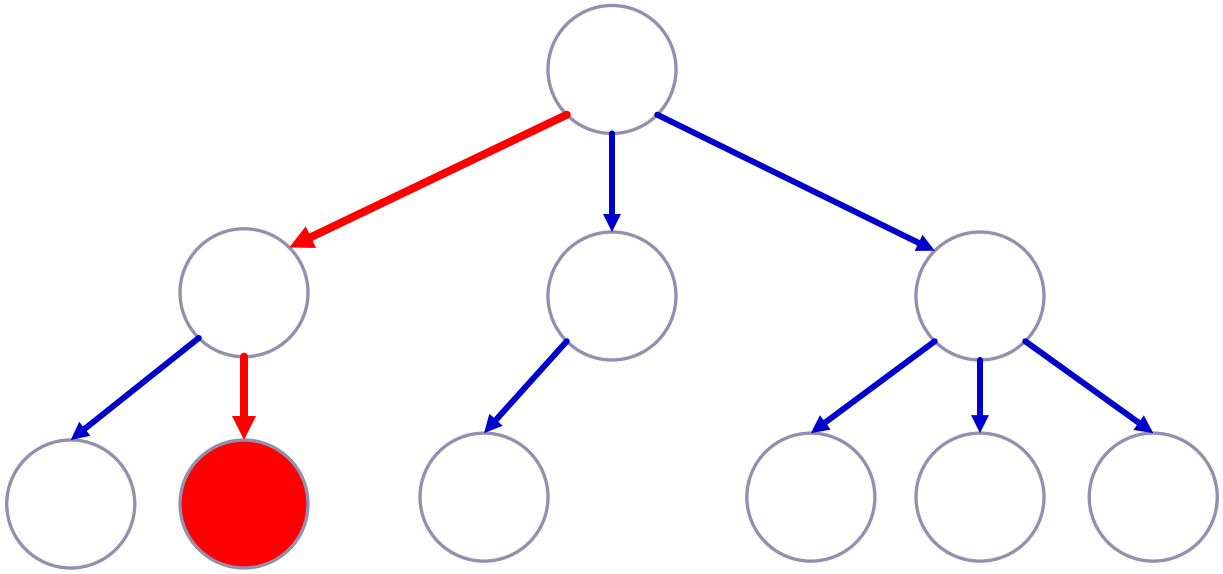
Step 1: Selection

Select a node that we will start the simulation from.



Step 1: Selection

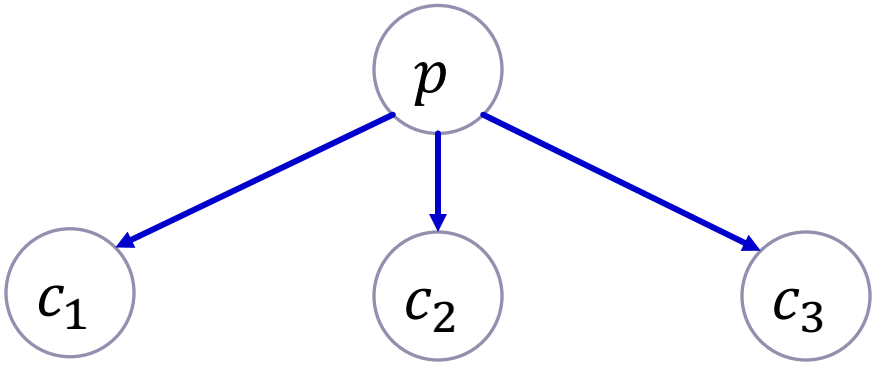
Select a node that we will start the simulation from.



How did we pick this path?

Step 1: Selection

Select a node that we will start the simulation from.



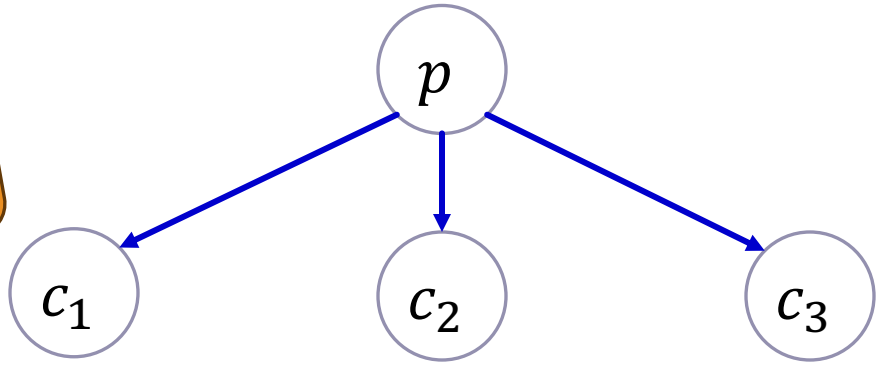
Pick a child node c_i of the parent node p, that maximizes

$$v_i + C \times \sqrt{\frac{\log n_p}{n_i}}$$

Step 1: Selection

Select a node that we will start the simulation from.

The UCB criterion



$$v_i + C \times \sqrt{\frac{\log n_p}{n_i}}$$

Value of c_i (prob. of winning). Should be in $[0,1]$

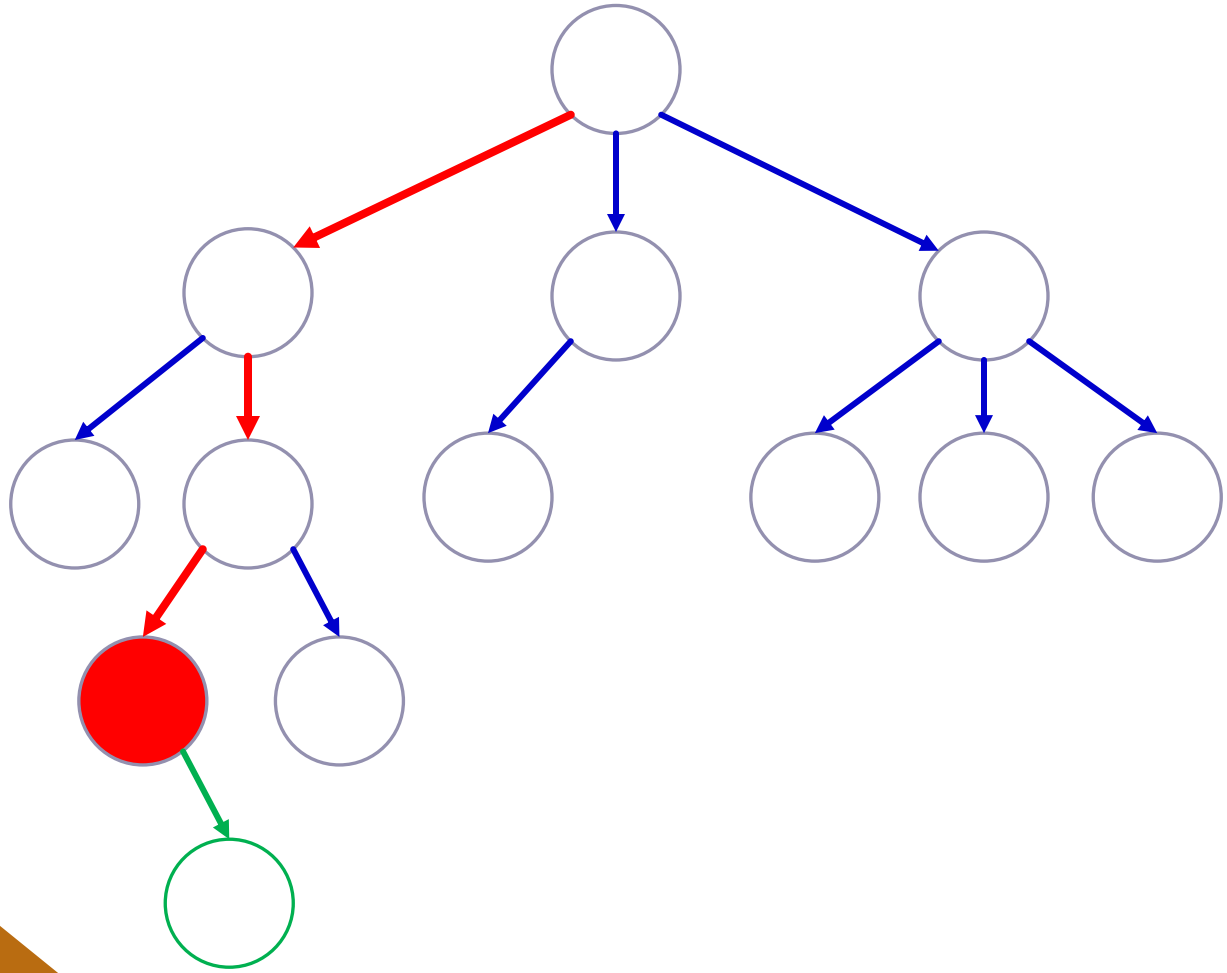
A tunable parameter

Number of times parent p was visited

Number of times child c_i was visited

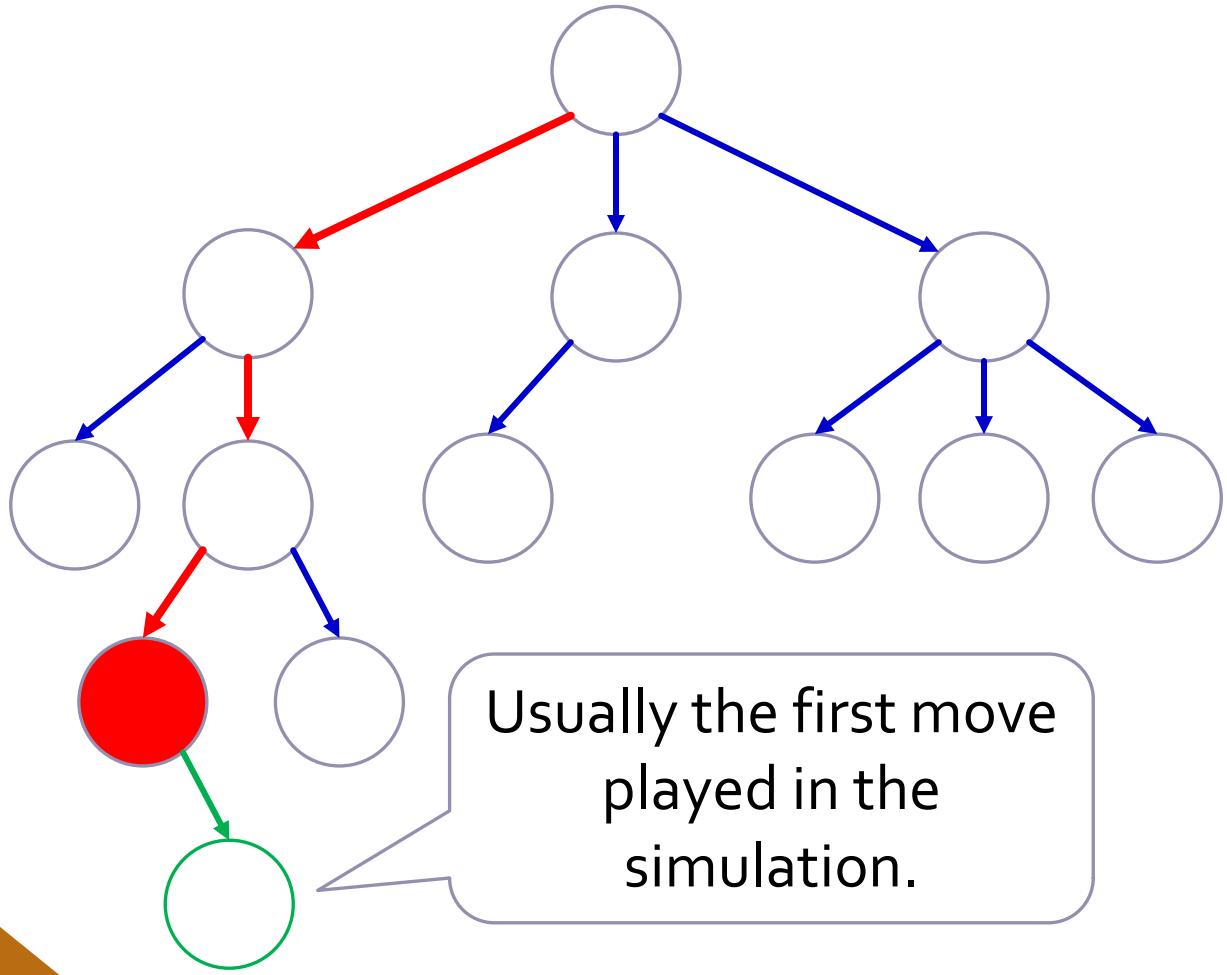
Step 2: Expansion

Add a node to the search tree.



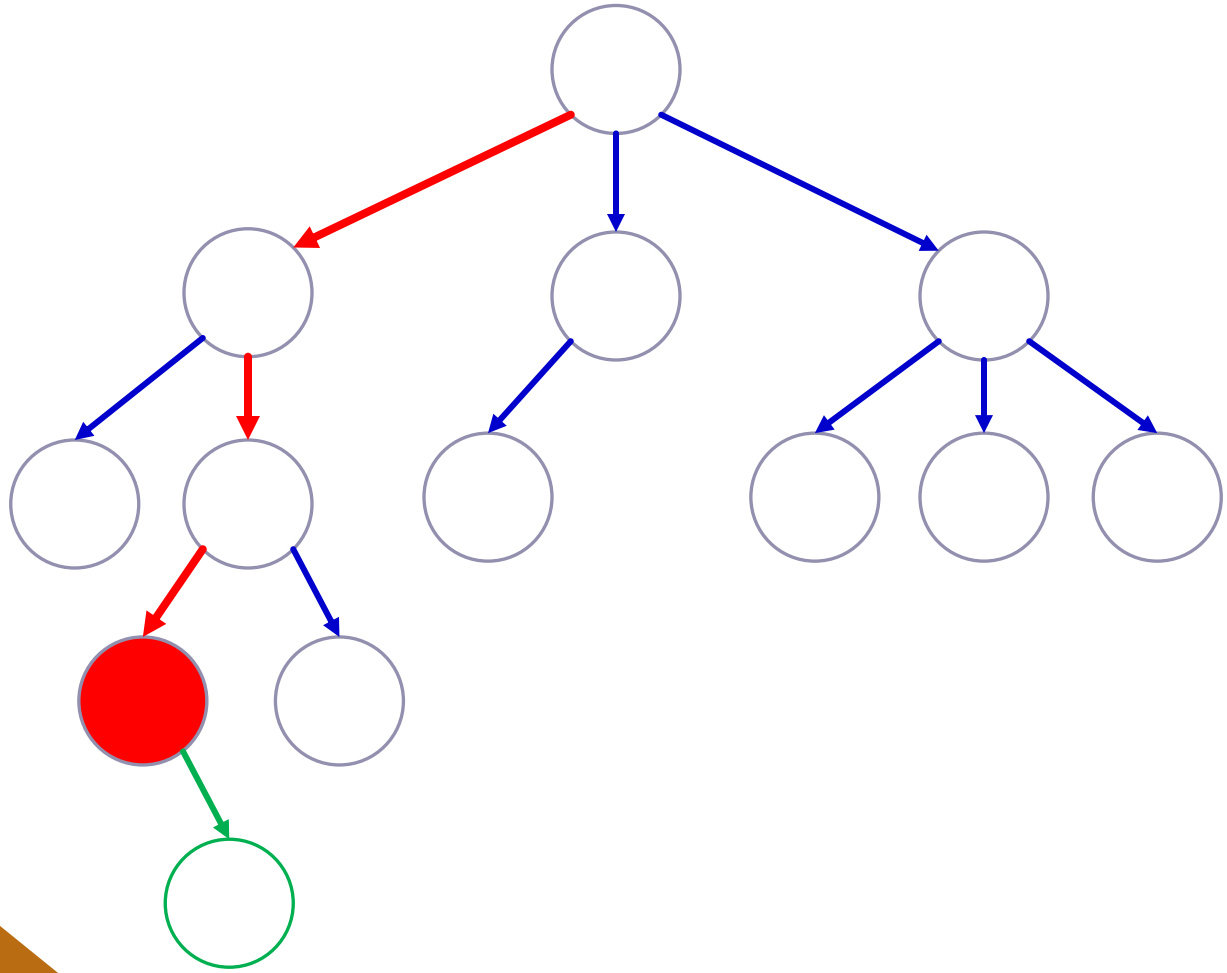
Step 2: Expansion

Add a node to the search tree.



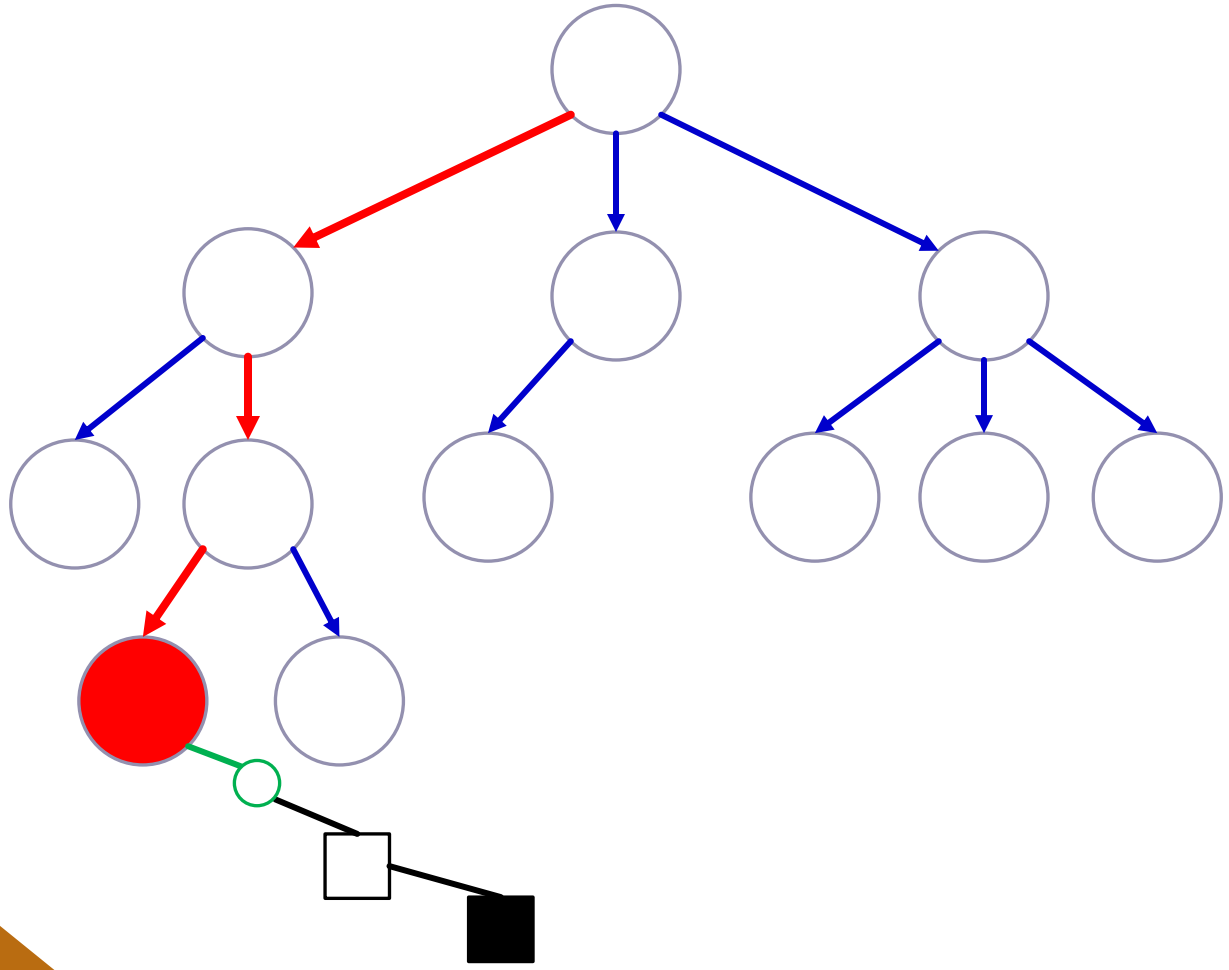
Step 3: Simulation

Simulate the game by letting players auto-play until the game ends.



Step 3: Simulation

Simulate the game by letting players auto-play until the game ends.



Step 3: Simulation

Simulate the game by letting players auto-play until the game ends.

The choice of moves that we let players execute during the simulation has **dramatic effect** on process quality.

Simulation strategy: what strategy should we let players execute? Random or… more refined?

E.g. in chess: greedily capture high-value pieces

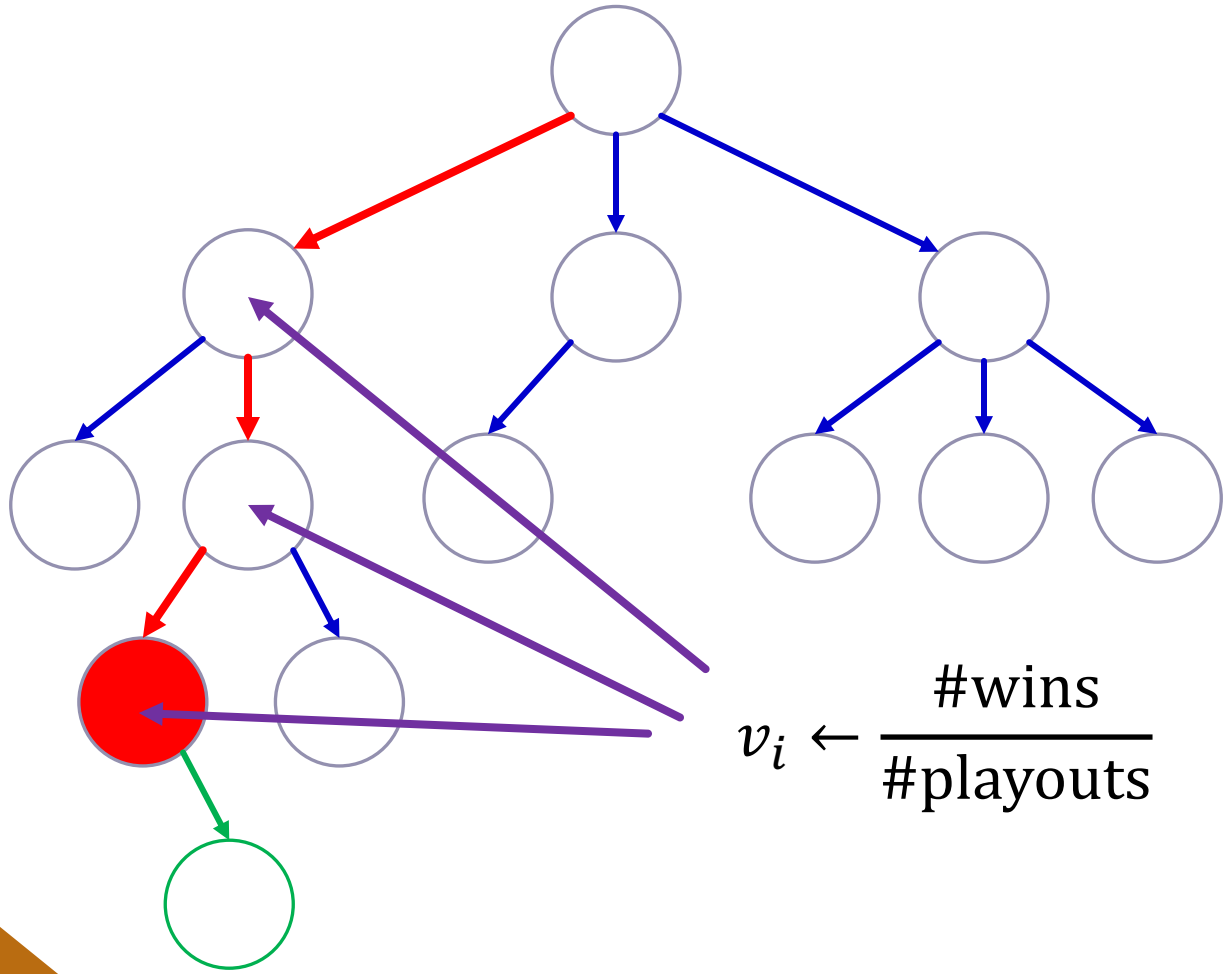
Step 3: Simulation

Simulate the game by letting players auto-play until the game ends.

Use heuristics/evaluations/deep learning to figure out what player moves result in good outcomes.

Step 4: Backpropagation

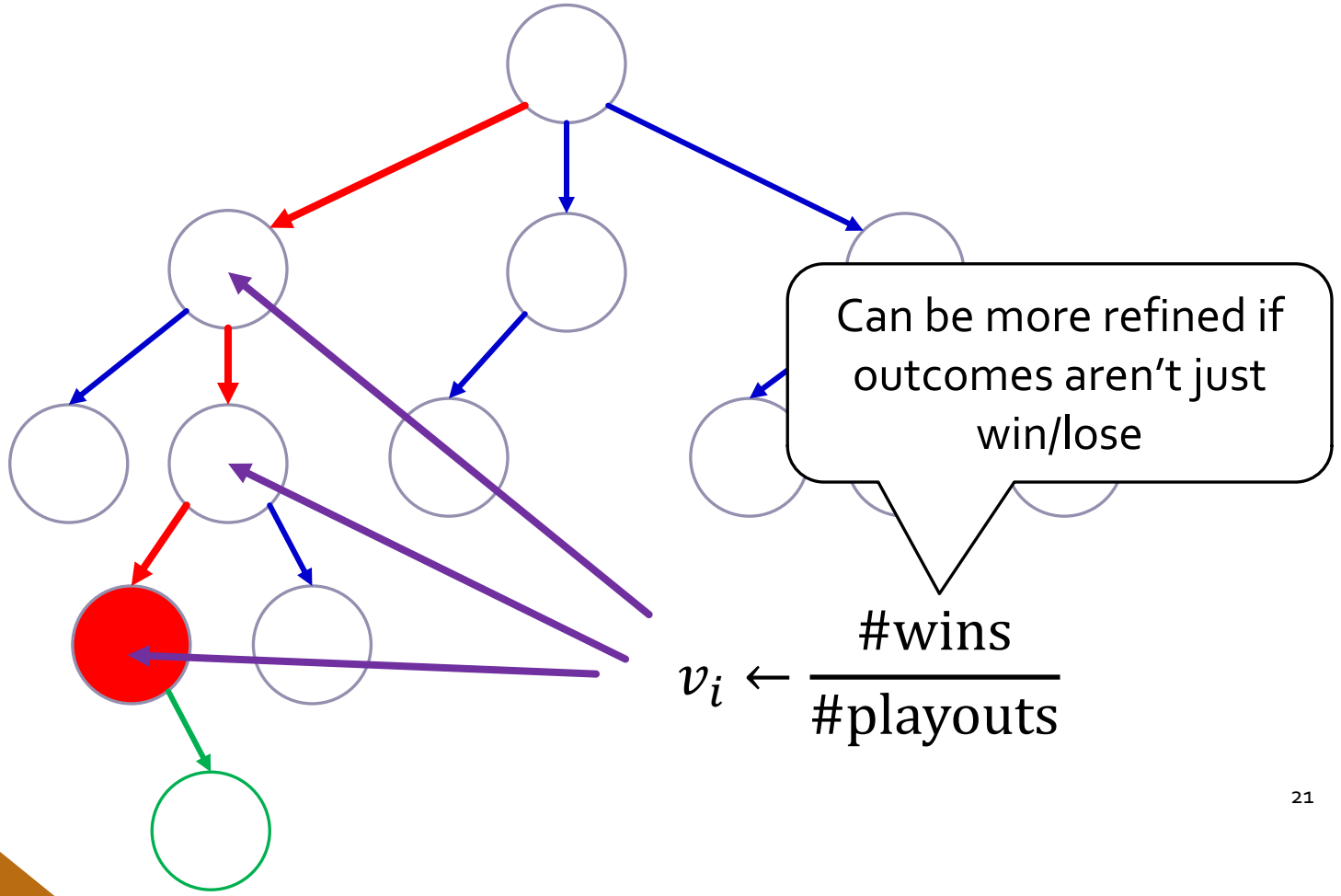
Update the values of nodes in the tree



$$v_i \leftarrow \frac{\#\text{wins}}{\#\text{playouts}}$$

Step 4: Backpropagation

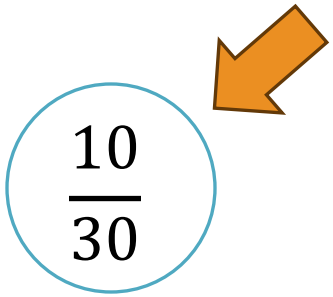
Update the values of nodes in the tree

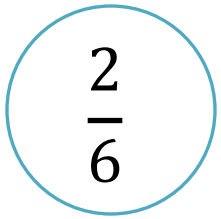


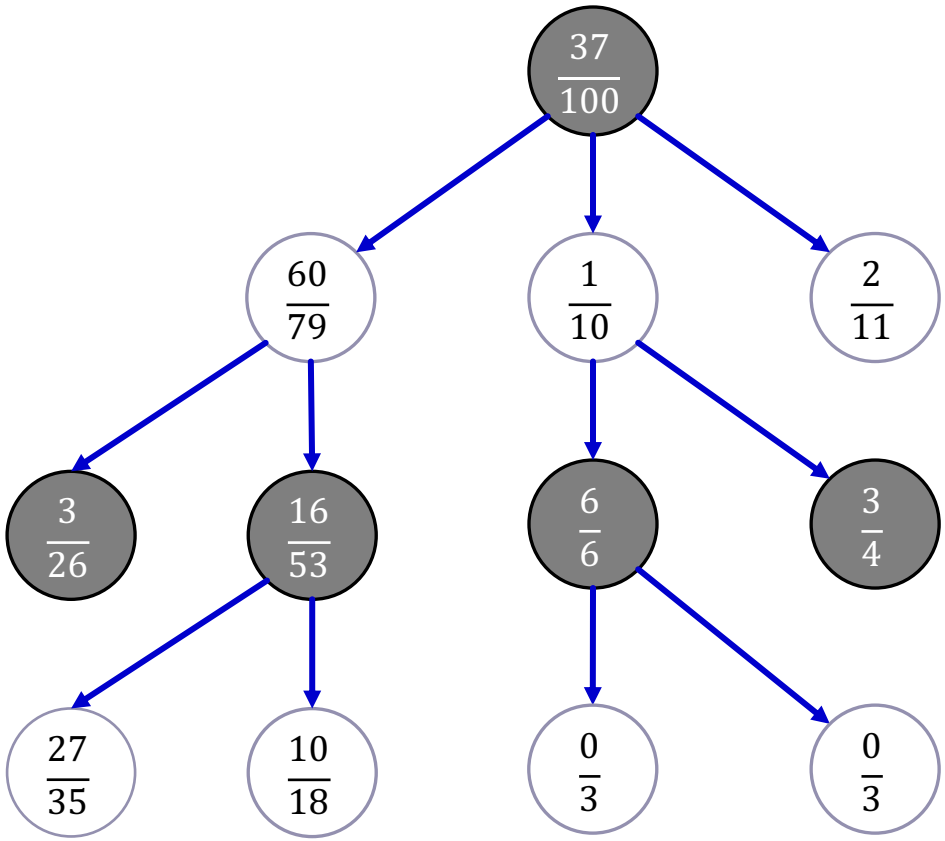
Return: the move that we played the largest number of times.

If we played a move often, it had a good enough win ratio to justify the many plays.

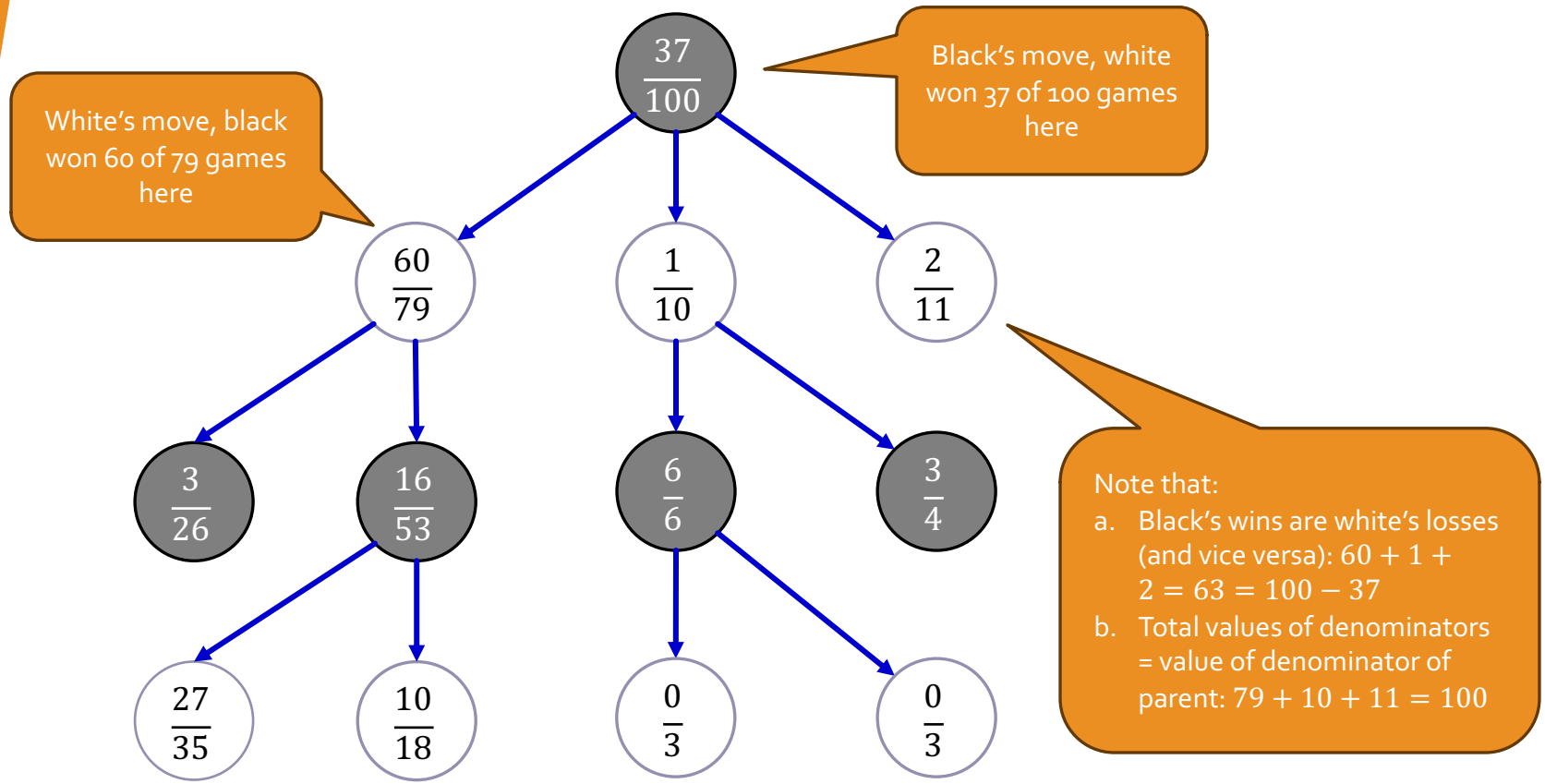
Some moves may have better win ratios but weren't played enough – **uncertainty!**



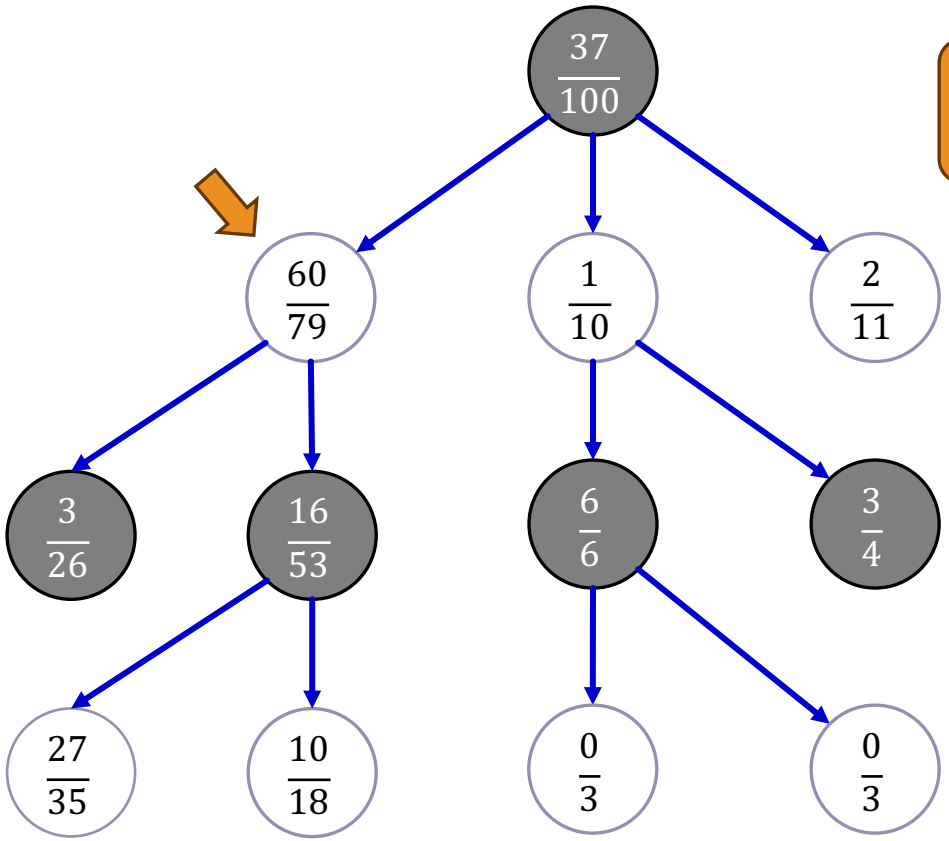




$$val = v_i + \sqrt{2} \times \sqrt{\frac{\log n_p}{n_i}}$$

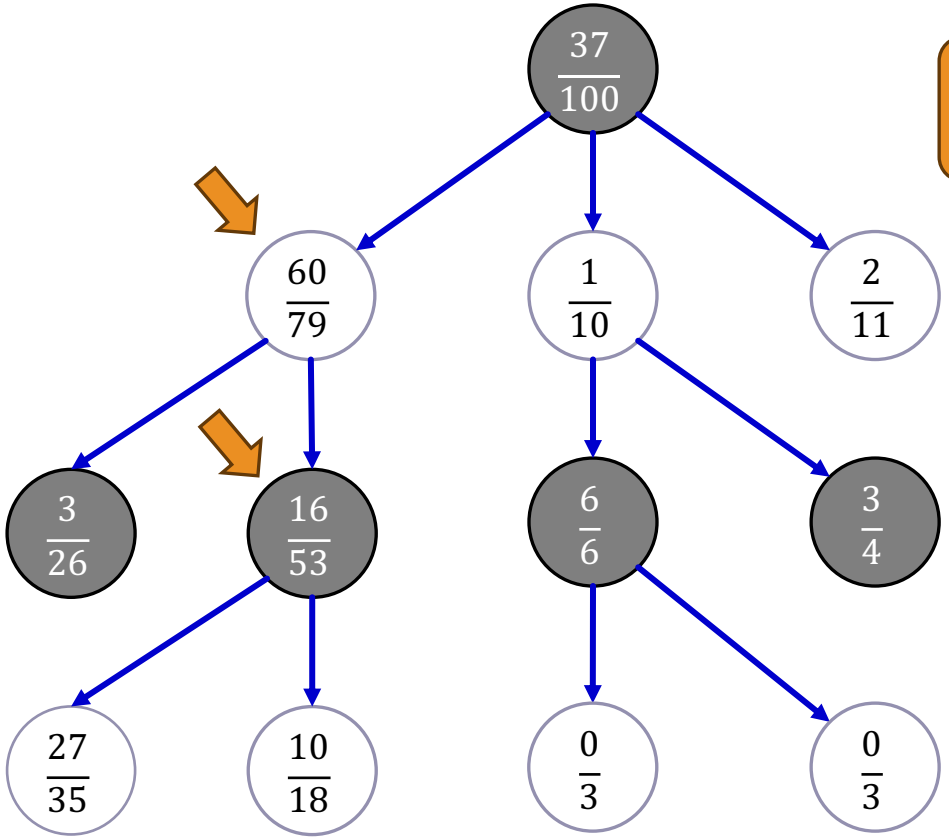


$$val = v_i + \sqrt{2} \times \sqrt{\frac{\log n_p}{n_i}}$$



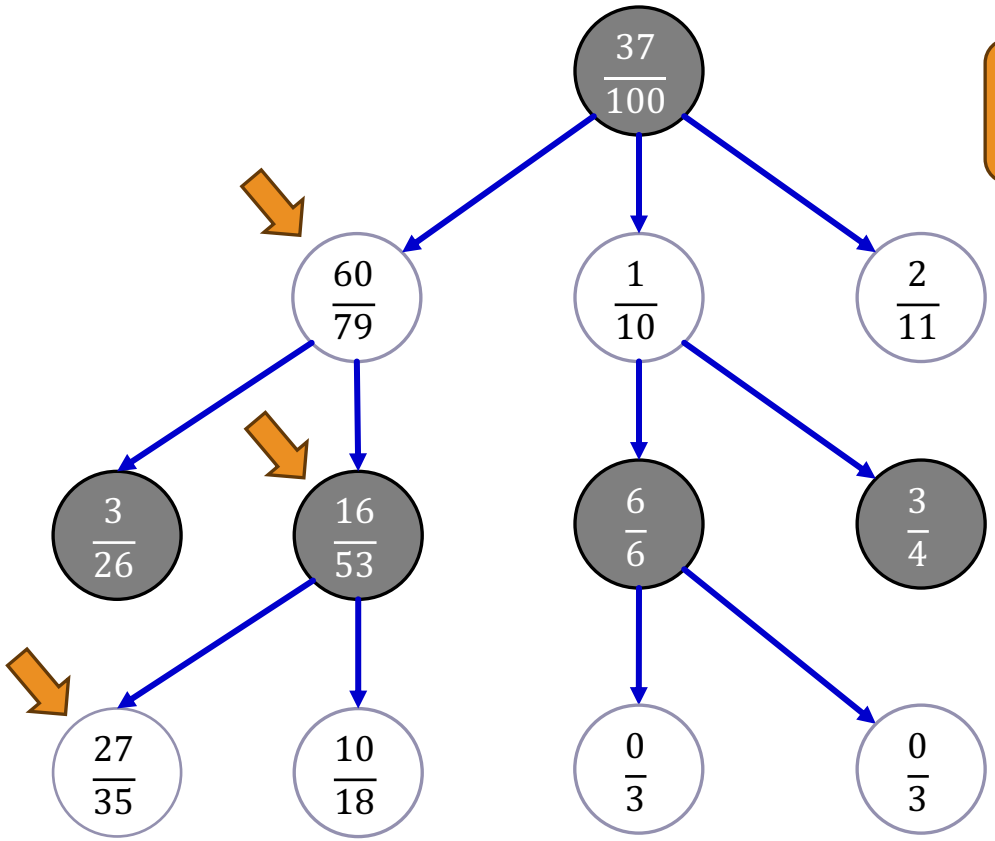
Step 1: selection

$$val = v_i + \sqrt{2} \times \sqrt{\frac{\log n_p}{n_i}}$$



Step 1: selection

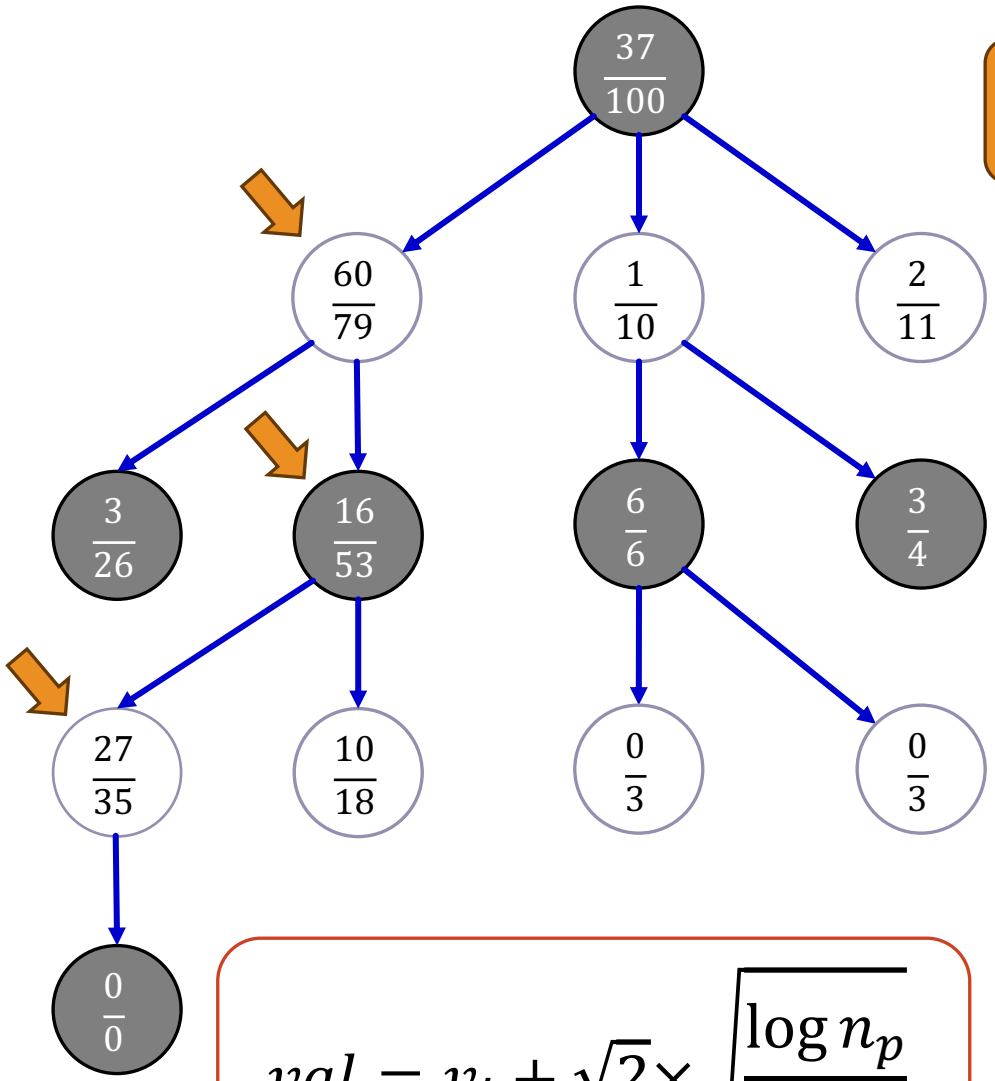
$$val = v_i + \sqrt{2} \times \sqrt{\frac{\log n_p}{n_i}}$$



Step 1: selection

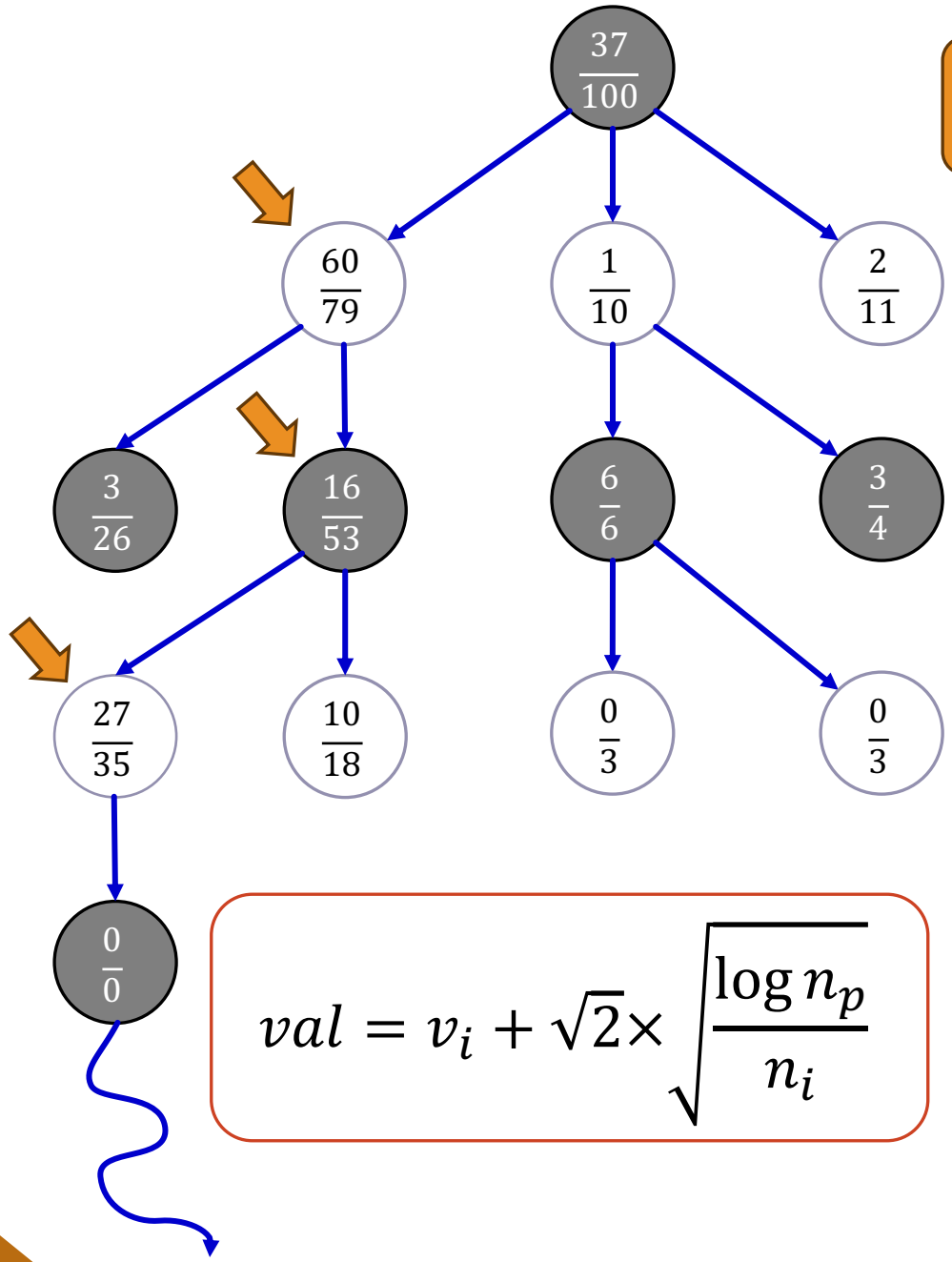
$$val = v_i + \sqrt{2} \times \sqrt{\frac{\log n_p}{n_i}}$$

Step 2: expansion



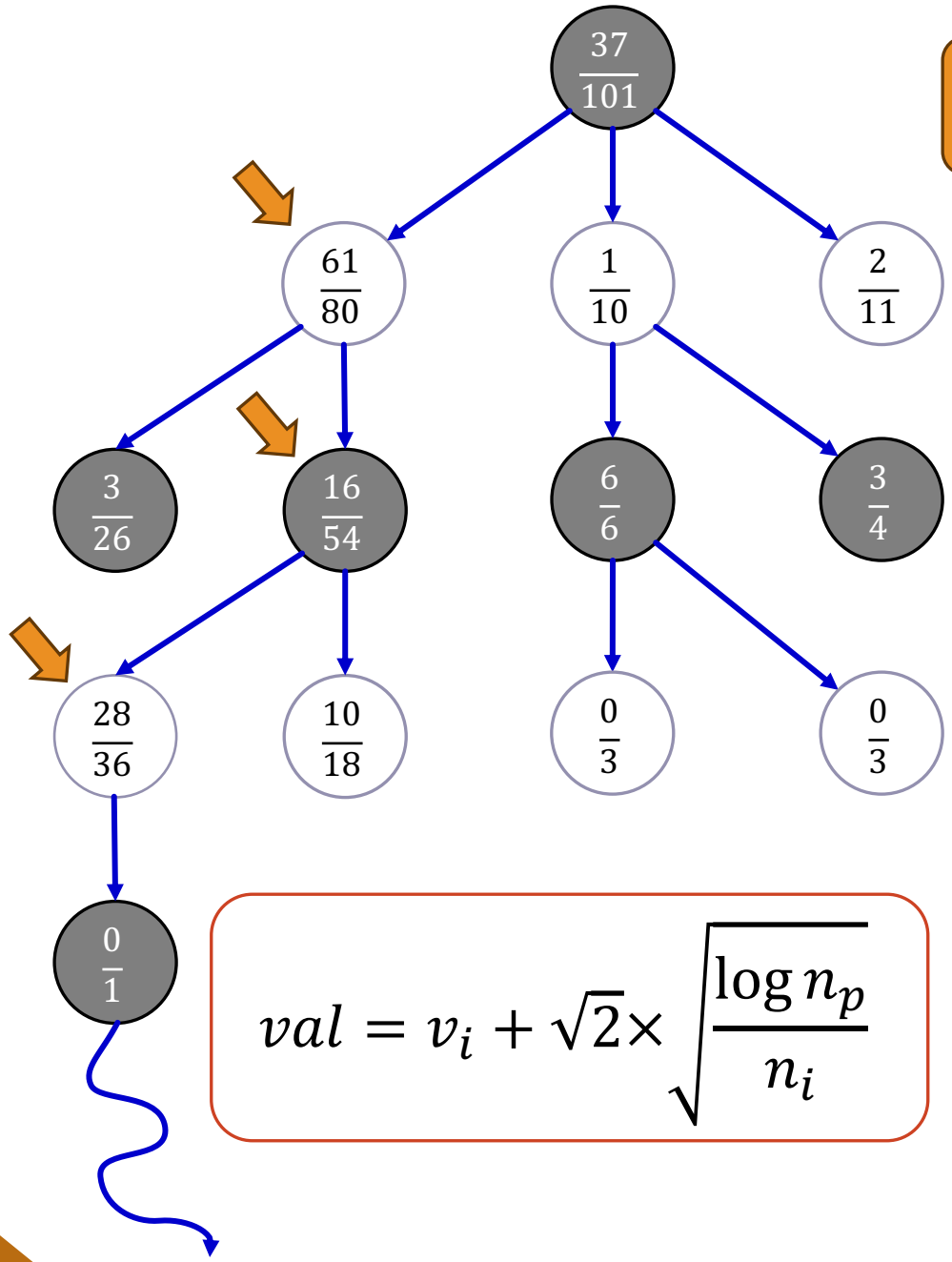
$$val = v_i + \sqrt{2} \times \sqrt{\frac{\log n_p}{n_i}}$$

Step 3: simulation

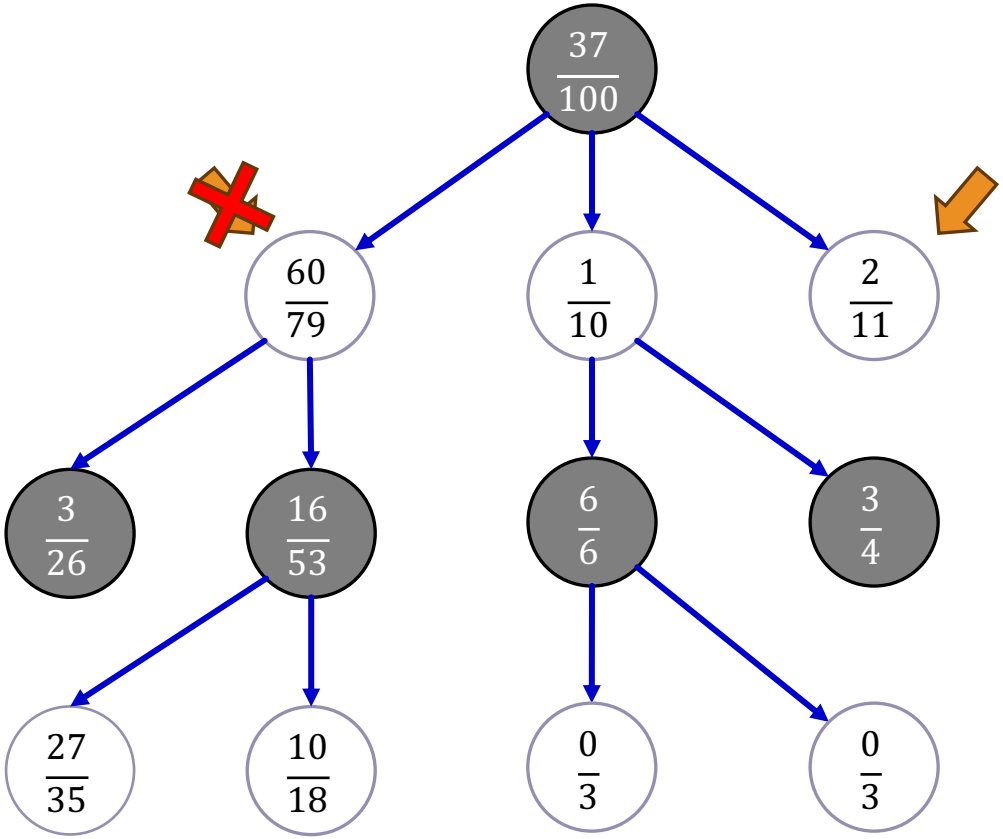


$$val = v_i + \sqrt{2} \times \sqrt{\frac{\log n_p}{n_i}}$$

Step 4: backpropagation



$$val = v_i + \sqrt{2} \times \sqrt{\frac{\log n_p}{n_i}}$$



$$val = v_i + 1.5 \times \sqrt{\frac{\log n_p}{n_i}}$$

Comparing Approaches

We have a game whose branching factor is 32, and the average game has 100 moves.

We have the compute capacity to evaluate a billion (10^9) game states before we can make a move.

How deep can we search?

- **Minimax search:** solve $32^k = 10^9$ to get $k \simeq 6$
- **Alpha-Beta pruning:** optimal ordering doubles the depth to 12.
- **Monte-Carlo:** can run $\frac{10^9}{100} = 10^7$ simulations before deciding.

Conventional wisdom: MCTS is better when branching factor is high or when it is hard to define a good evaluation function.

- **No prior knowledge :** selection strategies can be learned via simulations (e.g. using deep learning).
- **Missing key moves:** if there is a single move that wins the game, MCTS may miss it.
- **Missing obvious wins:** some winning moves can easily be found using minimax/alpha-beta, whereas MCTS may take a long time to find them.